

2024 年非专业级别软件能力认证第一轮

(CSP-S) 提高级 C++语言模拟试题

考生注意事项:

试题纸共有 12 页, 答题纸共有 1 页, 满分 90 分。请在答题纸上作答, 写在试题纸上的一律无效。

不得使用任何电子设备(如计算器、手机、电子词典等)或查阅任何书籍资料。

一、单项选择题(共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

1. 在 16 进制下, 在 $(100)_{16}$ 到 $(200)_{16}$ 之间, 有多少个数在把每一位都相加后再除以 $(F)_{16}$ 的余数为 9 (C)

A. 15 B. 16 C. 17 D. 18

2. A、B、C 三个社区需要建设若干个 5G 基站, 三个社区可供选择的建设基站地点分别有 2 个、4 个、5 个, 现从 A、B、C 三个社区分别选取 1、2、3 个地点随机分配给甲、乙、丙三个施工队进行建设, 要求每个施工队只能承接一个社区, 则承建方式有: (A)

A. 720 种 B. 480 种 C. 360 种 D. 120 种

3. 小明和姐姐用 2013 年的台历做游戏, 他们将 12 个月每一天的日历一一揭下, 背面粘上放在一个盒子里, 姐姐让小明一次性帮她抽出一张任意月份的 30 号或者 31 号。问小明一次至少应抽出多少张日历, 才能保证满足姐姐的要求? (C)

A. 346 B. 347 C. 348 D. 349

4. 在 Linux 系统终端中, 以下哪个命令用于复制文件? (A)

A. cp B. dd C. ll D. ls

5. 在程序设计中, `#include <stdio.h>` 的作用是什么? (A)

A. 包含输入输出函数的库
B. 包含标准数学库
C. 包含字符串处理库
D. 包含内存管理库

6. 在图的广度优先搜索 (BFS) 中, 常用的数据结构是: (B)

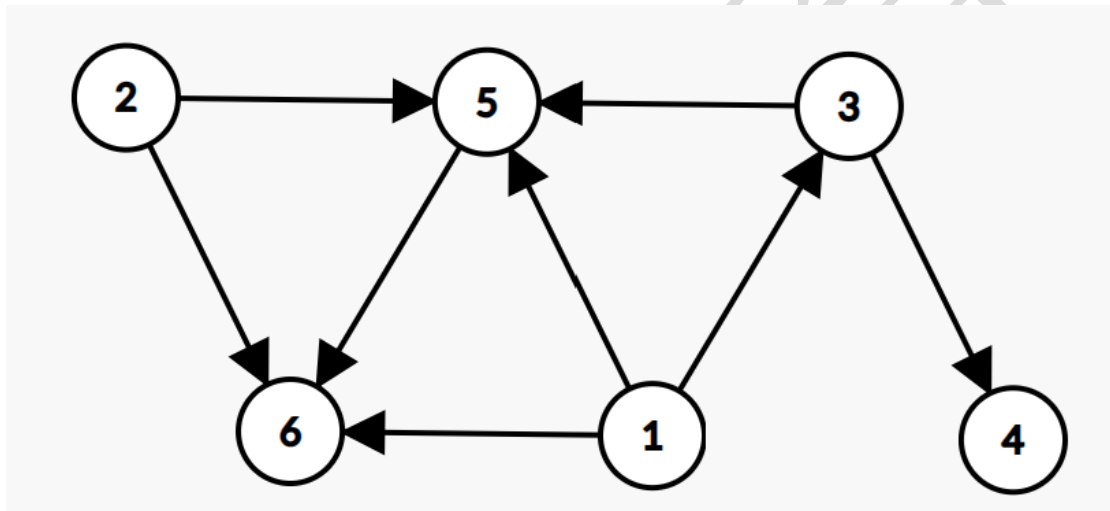
- A. 栈
 - B. 队列
 - C. 链表
 - D. 哈希表
- 7、在栈的实现中，哪个操作用于删除栈顶元素？（ B ）
- A. push()
 - B. pop()
 - C. top()
 - D. peek()
- 8、在 C 语言中，sizeof 操作符的功能是：（ A ）
- A. 计算变量的内存占用大小
 - B. 计算数组的长度
 - C. 获取变量的地址
 - D. 获取变量的类型
- 9、在 Linux 系统终端中，cat 指令的作用是？（ A ）
- A. 查看文件的内容
 - B. 查看网络设置
 - C. 下载网络文件
 - D. 删除目录
- 10、下列哪个算法不可以检查有向图中有无负权环？（ D ）
- A. 深度优先遍历 DFS
 - B. Bellman-Ford
 - C. SPFA
 - D. 拓扑排序
- 11、某公交站台附近区域停放 A 型共享单车 4 辆，B 型共享单车 5 辆，C 型共享单车 6 辆。一公交车到站后，下车的乘客随机选择其中 13 辆单车骑走。问 B 型和 C 型单车全部被骑走的概率在以下哪个范围内？（ A ）
- A. 在 10%以下
 - B. 在 10%~15%之间
 - C. 在 15%~20%之间

D. 超过 20%

12、若定义 `int a=8; int b=-8; int c=3 int d=-3;`则下列选项内所有计算结果都为-2 的是? (B)

- A. `a%c` 和 `a%d`
- B. `b%c` 和 `b%d`
- C. `a%d` 和 `b%c`
- D. `a%c` 和 `b%d`

13、如图是一张包含 6 个顶点的有向图，并且不存在环。如果对 6 个顶点进行拓扑排序，请问总共有多少不同的排序结果? (B)



- A. 1
- B. 18
- C. 36
- D. 360

14、对于下列排序算法的描述，不正确的是: (B)

- A. 计数排序的时间复杂度的关键是数字的数值范围而不是数字的数量。
- B. 基数排序的内层排序可以选择不稳定的排序算法。
- C. 桶排序的内层排序可以选择不稳定的排序算法。
- D. 希尔排序的内层排序可以选择不稳定的排序算法。

15、Alice 和 Bob 在玩一个游戏，游戏的规则是两人在连续的 n 个格子上画 $\times$ ，某个人画完之后使得有连续的 3 个格子上都被画了 $\times$ ，那么他获胜。当 n 等于多少时，先手玩家必胜 (A)

- A. 7
- B. 9
- C. 11
- D. 13

二、阅读程序（程序输入不超过数组或字符串定义范围；判断题正确填√，错误填×；出特殊说明外，判断题 2 分，选择题 3 分，合计 30 分）

(1)

```
01 #include<iostream>
02 #include<string>
03 using namespace std;
04 struct node {
05     int son[26];
06     int count;
07     int fa;
08     char c;
09 }list[400003];
10 int cnt = 2;
11 long long ans = 0;
12 int main() {
13     string s;
14     cin >> s;
15     int node = 1;
16     list[node].count = 1;
17     for (int i = 0; i < s.size(); i++) {
18         char c = s[i];
19         if (list[node].c == c) {
20             node = list[node].fa;
21         }
22         else
23         {
24             if (list[node].son[c - 'a'] == 0) {
25                 list[node].son[c - 'a'] = cnt;
26                 list[list[node].son[c - 'a']].fa = node;
27                 cnt++;
```

```

28         }
29         node = list[node].son[c - 'a'];
30         list[node].c = c;
31     }
32     list[node].count++;
33     ans += list[node].count - 1;
34 }
35 cout << ans;
36 return 0;
37 }

```

假设输入仅一行，是一个长度在 4×10^5 以内，只有小写字母的字符串，完成下面的判断题和单选题：

单选题

16. 把 16 行的 1 改成 0，对于同样的输入，输出结果一定比原来小。（ × ）
17. 当输入的是 “accabccb” 时，输出的是 4。（ × ）
18. 当输入的是偶数个相同的字符，输出的数字也一定是偶数。（ × ）

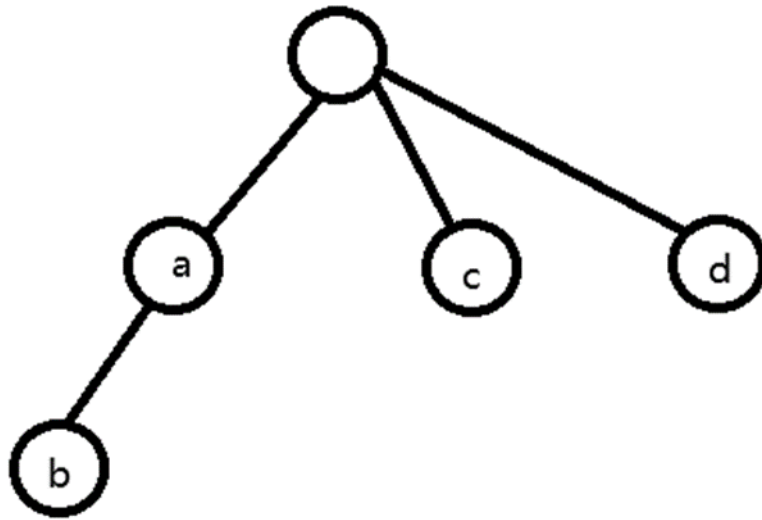
单选题

19. 当输入的字符串是 “abcdefghijklmn”，输出的结果是（ A ）
A. 0 B. 1 C. 13 D. 14
20. 当输入的字符串是 “abbacddcc”，在程序执行到第 35 行是，cnt 的值是（ C ）
A. 4 B. 5 C. 6 D. 7
22. 当输入 n 个一样的小写字母，输出的结果和 n 的公式是（ D ）
A. n B. n^2 C. $n^{2/4}$ D. $\lfloor n/2 \rfloor \lfloor n/2 \rfloor$

字典树求解 CSP2023-S 消消乐（做此题可以硬模拟）

16. × 不一定，当输入的所有字母的都不相同时，list[1].count 少一，不会有执行当 33 行的时候，node=1；
17. × 5
18. × 当 n 只能被 2 整除，不能被 4 整除，输出的数字是奇数。
19. A

20.C 创建的树如下图，总共有 5 个节点，又由于是 cnt 先用后加，所以是 $5+1=6$ ；
(根节点由 10 行 15 行均可知是从 1 开始)



21.D 当输入的小写字母一样时，相当于是求有多少种长度为偶数的子串的取法。(这个两个括号分别表示向上取整和向下取整)

(2)

```
01 #include <iostream>
02 #include <string>
03 #include <cmath>
04 #include <iomanip>
05 const int N = 50;
06 const int L = 3000;
07 using namespace std;
08 int n, a[N], sum;
09 double ans;
10 bool f[L][L];
11
12 bool check(int x, int y, int z)
13 {
14     if (x + y > z && x + z > y && y + z > x) return 1;
```

```

15     return 0;
16 }
17
18 double work(double x, double y, double z)
19 {
20     double p = (x + y + z) / 2;
21     return sqrt(p * (p - x) * (p - y) * (p - z));
22 }
23
24 int main()
25 {
26     int i, j, k;
27     cin >> n;
28     for (i = 1; i <= n; i++) {
29         cin >> a[i];
30         sum += a[i];
31     }
32     f[0][0] = 1;
33     for (k = 1; k <= n; k++)
34         for (i = sum; i >= 0; i--)
35             for (j = sum ; j >= 0; j--)
36                 {
37                     if (i - a[k] >= 0 && f[i - a[k]][j]) f[i][j] = 1;
38                     if (j - a[k] >= 0 && f[i][j - a[k]]) f[i][j] = 1;
39                 }
40     ans = -1;
41     for (i = sum ; i > 0; i--)
42         for (j = sum ; j > 0; j--)
43             {
44                 if (!f[i][j]) continue;
45                 if (!check(i, j, sum - i - j)) continue;
46                 ans = max(ans, work(i, j, sum - i - j));
47             }
48     if (ans != -1) cout << fixed << setprecision(2) << ans << endl;
49     else cout << ans << endl;

```

```
50     return 0;
```

```
51 }
```

假设输入的所有数字都是大于 2 小于 50 的整数，完成下列判断题和选择题：

判断题

22. 输出的结果一定是一个保留 2 位小数的数字 (×)

23. 当输入 “6 3 3 4 4 5 5” 时，输出的结果是 “24.00” (×)

24. 把 34 行和 35 行的 “sum” 改成 “sum/2”，输出的结果不变 (✓)

选择题

25. 当输入 “3 8 6 10” 时，输出的结果为 (D)

A. -1 B. 5.00 C. 6.00 D. 24.00

26. 当输入 “3 3 4 12” 时，输出的结果为 (A)

A. -1 B. 5.00 C. 6.00 D. 24.00

27. 当输入 “5 5 5 5 6 7”，输出的结果为 (D)

A. 26.66 B. 27.50 C. 32.80 D. 34.29

输入 n 个数字，把 n 个数字分成三组，每组的总和构成三角形的三条边。使得三角形面积尽可能的大。输出目标三角形的面积并保留两位小数。整体思路的动态规划 $f[i][j]$ 表示 n 个数字能分出两堆不同的 i 和 j

22. × 当不能构成三角形时，输出-1

23. × 组成 3 个 8 的等边三角形面积最大。(6 8 10 直角三角形的面积是 24) 27. 71

24. ✓ 因为三角形的最大边之和要小于两个短边之和，所以任意一个边都不能超过总长度的 $1/2$

25. D

26. A

27. D (7 10 11 的面积最大) 带入海伦公式计算即可。

三、完善程序 (单选题，每小题 3 分、合计 30 分)

1. (希尔排序) 输入数组的长度 n 和 n 个整数，对其进行排序后输出。

试补完程序。

```
01 #include<iostream>
```

```
02 int a[10000];
```



```

03 void shell_sort(int array[], int length) {
04     int h = ①;
05     while (h >= 1) {
06         for (int i = h; i < length; i++) {
07             for (int j = i; j >= h && array[j] < array[j - h]; ②) {
08                 array[j] += array[j - h];
09                 array[j - h] = array[j] - array[j - h];
10                 ③;
11             }
12         }
13         ④;
14     }
15 }
16 int main() {
17     int n = 0;
18     std::cin >> n;
19     for (int i = 0; i < n; i++) std::cin >> a[i];
20     shell_sort(⑤);
21     for (int i = 0; i < n; i++) std::cout << a[i] << " ";
22     return 0;
23 }

```

1. ①处应填(C)

A. 0xffffffff B. 1 C. length/2 D. INT32_MAX

2. ②处应填(B)

A. j += h
 B. j -= h
 C. j++
 D. j--

3. ③处应填(A)

A. array[j] -= array[j - h]
 B. array[j] += array[j - h]
 C. array[j - h] -= array[j]

D. `array[j - h] += array[j]`

4. ④处应填(B)

- A. `h--`
- B. `h/=2`
- C. `h/=3`
- D. `h/=4`

5. ⑤处应填(B)

- A. `0, n-1`
- B. `0, n`
- C. `1, n`
- D. `1, n+1`

1. C A 选项错误外, BD 都是由于题目规定用希尔排序不能选(代码不会出问题但是效率低)

2. B 从 for 循环前面从 `j>=h` 可知循环是从后往前

3. A 这三行代码是做 `array[j]` 和 `array[j-h]` 交换

4. B 希尔排序只有不断/2 才能保证 `h>=1` 的 while 循环条件退出

5. B 从 06 行可以看出传入的第二个参数是开区间右端点 07 行 `j>=h` 和 `j-h` 表示是闭区间左端点

2. (背包问题的前 K 优解) 考虑在有 N ($1 \leq N \leq 500$) 种物品放在 V ($1 \leq V \leq 5000$) 容量的背包中, 每一种物品都有给定体积和价值 (不超过 1000 的正整数), 一种物品只有一件, 总共有 2^N 种方案。对于不同的选择方案, 要求从大到小依次输出前 K ($1 \leq K \leq 50$) 大的总价值, 如果方案不足 K 个则输出所有方案的结果。试补全动态规划算法。

```
01 #include<iostream>
02 #include<cstring>
03 using namespace std;
04 int f[5010][51], ans = 0;
05 int k, v, n, w[501], c[501], t[51];
06 int main() {
07     memset(f, ①, sizeof(f));
08     cin >> k >> v >> n;
```

```

09     for (int i = 1; i <= n; i++)
10         cin >> w[i] >> c[i];
11     f[0][1] = 0;
12     for (int i = 1; i <= n; i++)
13         ② {
14             int t1 = 1, t2 = 1, t[51], len = 0;
15             while (t1 + t2 <= ③) {
16                 if (f[j][t1] > f[j - w[i]][t2] + c[i])
17                     ④;
18                 else ⑤;
19             }
20             for (int K = 1; K <= k; K++) f[j][K] = t[K];
21         }
22
23     for (int i = 1; i <= k; i++) if (f[v][i] > 0) cout<< f[v][i] <<
endl;
24     return 0;
25 }

```

1. ①处应填(B)

- A. 0
- B. 128
- C. 256
- D. 255

2. ②处应填(C)

- A. for (int j = w[i]; j < v; j++)
- B. for (int j = 1; j <= v; j++)
- C. for (int j = v; j >= w[i]; j--)
- D. for (int j = v; j >= 1; j--)

3. ③处应填(B)

- A. k
- B. k + 1
- C. n
- D. f[S] = n + 1

4. ④处应填(C)

- A. `t[len++] = f[j][t1++]`
- B. `t[len++] = f[j][++t1]`
- C. `t[++len] = f[j][t1++]`
- D. `t[++len] = f[j][++t1]`

5. ⑤处应填(C)

- A. `t[len++] = f[j - w[i]][t2++] + c[i]`
- B. `t[len++] = f[j - w[i]][++t2] + c[i]`
- C. `t[++len] = f[j - w[i]][t2++] + c[i]`
- D. `t[++len] = f[j - w[i]][++t2] + c[i]`

`f(j)=max(f(j), f(j-v[i])+c[i])`

转移是相同的（这里是压掉了物品的那一维），题目需要获取前 K 优方案，则需要另外加一维记录前 K 优的方案，在 `f(j)` 已经排好序的情况下，对 `f(j)` 和 `f(j-v[i])+c[i]` 做二路归并排序即可。

1. B A 和 C 选项的效果相同，数组全部初始化为 0，D 初始化的值为 -1（-1 的补码即为若干个 1），只有 B 选项会是一个较小的负数。

2. C 压掉物品那一维，体积维必须从高到低（如果大于 `w[i]` 则会数组越界）

3. B 第 14 行处可知进行归并排序的两个下标都是从 1 开始，那么 `t1+t2-2` 等于已经选了的数字，循环需要选 k 个数（`t1+t2 < k+2` 比较好理解，但题目里是小于等于）

4. C 5. C 由 14 行可知 `t1t2` 从 1 开始 `len` 从 0 开始，由 20 行可知前 k 优个解的下标也是从 1 开始。所以都是 `++len`，`t1/2++`